DEVOPS LEARNING PROJECT · V0.0.1

Weather data. Production thinking.

HiveBox turns openSenseMap sensor readings into a searchable web experience, then wraps it in caching, background storage, containers, health checks, CI and Kubernetes.

VIEW GITHUB REPOSITORY ↗

SEE THE WORKFLOW ↓

4
LOCAL SERVICES

3
OPERATIONAL ENDPOINTS

1
REUSABLE APP IMAGE

01 / DATA PATH

From place name to observation.

The API accepts a location instead of forcing users to know sensor IDs. It resolves the place, finds nearby openSenseMap stations, normalizes their temperature measurements and returns a small, consistent response.

01

User searches

The browser sends a country, city or region to the Flask API.

02

Resolve bounds

Nominatim translates that name into a geographic bounding box.

03

Fetch sensors

openSenseMap returns readings found inside those bounds.

04

Normalize

The API handles temperature labels, units and invalid readings.

05

Respond

The frontend renders the latest usable observations and summary.

Valkey first

Repeated searches use the five-minute cache. A cache miss triggers the upstream APIs and stores the new response.

LESS LATENCY · LESS LOAD

02 / RUNTIME

One image. Separate jobs.

Docker builds the Python application once. Compose or Kubernetes starts that same image as an HTTP API and as a periodic worker, alongside Valkey for caching and MinIO for object storage.

TRAFFIC

Browser / client

Calls the public API through port mapping locally or Ingress in Kubernetes.

↓

SAME IMAGE

API + worker

Gunicorn serves Flask. A second process runs `python main.py worker`.

WORKER CYCLE
fetch → cache → archive every 5 min

↓

STATE

Valkey + MinIO

Valkey holds short-lived JSON. MinIO keeps timestamped snapshots outside the app container.

GET /healthz

Is the process alive? No dependency checks.

GET /readyz

Can it safely receive traffic under the project’s majority-and-cache rule?

GET /metrics

Expose request and process signals for monitoring.

03 / DELIVERY

Prove it, then ship it.

Every push follows the same quality gate. Local Compose makes the full system easy to reproduce; Kind tests orchestration concepts before the image is deployed to a public container platform.

01 **Lint + test**
Ruff checks Python and Pytest protects API behavior.

02 **Scan manifests**
Terrascan catches risky Kubernetes configuration.

03 **Build image**
CI creates exactly what will run in production.

04 **Smoke test**
A real container starts and must answer `/healthz`.

PUBLIC DEPLOYMENT

Keep the architecture; swap the state.

Run the image on a managed container service. Replace local Valkey and MinIO with managed Redis-compatible cache and S3 object storage. Keep secrets private and place HTTPS in front.

APPLICATION	container service
CACHE	managed Valkey/Redis
ARCHIVE	S3-compatible bucket
EDGE	DNS + TLS + rate limits